

JAI-1: A Thai-Centric Large Language Model

Attapol T. Rutherford^{1,2}, Jullajak Karnjanaekarin¹,
Narongkorn Panitsrisit¹, Pontakorn Trakuekul¹,
Sumana Sumanakul¹, Natchanon Pollertlam¹, and Upstage Team³

¹Jasmine Technology Solution, Thailand

²Department of Linguistics, Chulalongkorn University, Thailand

³Upstage, South Korea

`jts.ai.team@gmail.com`

Abstract

This technical report introduces JAI-1, a Thai-centric language model with 75B parameters. Recent Thai models have primarily relied on existing open-source models, applying additional training without structural modifications to specialize in Thai. However, this approach risks eroding pre-existing knowledge in the model’s parameter space during the injection of Thai-specific information, as optimized parameters for general tasks may conflict with new linguistic requirements. In contrast, JAI-1 adopts a upscaling strategy: starting from a smaller, high-performing English open-source LLM, we expanded its parameter space and utilized the newly allocated capacity to systematically integrate Thai-language knowledge. This methodology not only preserves the original model’s general intelligence but also establishes a unique architecture distinct from other open-source models, enabling scalable future enhancements.

During pre-training, JAI-1 was exposed to 1.5T tokens, including over 300B Thai language tokens. This was followed by post-training stages—supervised fine-tuning and alignment tuning—using more than 600K instruction-based examples. The final model demonstrated superior performance compared to Typhoon2-70B on Thai-centric benchmarks (IFEval-TH, MT-Bench-TH, and JAI-Hall-Bench), validating the efficacy of its upscaling and knowledge-integration framework.

Contents

1	Introduction	4
2	JAI-1 Overview	5
2.1	Upscaling Strategy for Architecture Design	5
2.2	Training Pipeline	6
3	JAI-1 Architecture	7
3.1	Upscaling Strategies	7
3.1.1	Tokenizer Adaptation	7
3.1.2	Advanced Depth-Up-Scaling	8
3.1.3	Mixture of Expert Design	11
3.2	Architecture Configuration	11
4	Pre-training	12
4.1	Thai Corpora for pre-training	12
4.2	Pre-training Strategy	14
5	Pre-trained LLM Evaluation	15
5.1	Evaluation Benchmarks for Pre-trained Models	15
5.2	Evaluation Results on Pre-trained Models	15
6	Post-training: Supervised Finetuning	16
6.1	Eng-Thai Translator and Thai Fluency Reward Model	17
6.1.1	Eng-Thai Translator	17
6.1.2	Thai Fluency Reward Model	17
6.2	Supervised Finetuning Data	17
6.2.1	General Chat Ability	18
6.2.2	Instruction Following	18
6.2.3	Wild Task Ability	18
6.2.4	Thai Cultural Understanding	19
6.2.5	Thai Safety	19
6.2.6	Long Context Understanding	19
6.3	Supervised Finetuning Stages	19
7	Post-training: Alignment-tuning	20
7.1	Alignment-tuning Method	21
7.2	Alignment-tuning Data	21
7.2.1	General chat alignment	21
7.2.2	Instruction following alignment	21
7.2.3	Hallucination alignment	22
8	Evaluation for Instruct Models	22
8.1	Evaluation Benchmarks for Instruct Models	22
8.2	Evaluation Results on Instruct Models	23

1 Introduction

Large Language Models (LLMs) have rapidly become ubiquitous in daily life, transforming how individuals access, organize, and leverage information to address everyday challenges. These models synthesize vast global knowledge, offering profound assistance in human activities. However, their continued advancement hinges on two critical factors: a deep understanding of linguistic and cultural contexts across diverse regions [1], and the development of resource-efficient architectures tailored to dynamic, real-world applications [2].

Despite the existence of over 7000+ languages worldwide, the majority of training data remains concentrated in English, creating significant disparities in representation [3]. This imbalance is particularly evident in Thailand, where linguistic and cultural nuances are underrepresented in global datasets [4]. Consequently, existing LLMs often struggle to capture the unique characteristics of Thai language and culture [5], limiting their effectiveness in localized contexts.

To bridge this gap, Thai-centric LLMs such as Typhoon [6, 7] and OpenThaiGPT [8] have recently emerged. Typhoon1 (7B) [6] adapted Mistral-7B through additional pre-training on a Thai corpus, evolving into Typhoon 1.5 [9] built on Qwen 1.5 [10], and the latest Typhoon 2 [7], leveraging Llama 3.1 with Thai data injection. Similarly, OpenThaiGPT [8] began by fine-tuning Llama 2, and its recent v1.5 series utilizes Qwen 2.5 checkpoints tuned on multi-million Thai instruction pairs. These Thai-centric LLMs follow an open-source paradigm: injecting proprietary Thai data into advancing backbones—an efficient community-driven strategy. However, this model construction risks eroding pre-existing knowledge in the model’s parameter space—catastrophic forgetting—when specializing for Thai [11, 12]. Additionally, lack of visibility into upstream architectures and data constrains transparency, auditing capacity, and bias control.

Building upon this momentum, our project also develops a Thai-centric LLM, named JAI-1(75B), designed with two considerations in mind. First, we utilize trillion-level pretraining data: a mix of English and Thai corpora, where English maintains and enhances general capabilities while Thai injects localized knowledge. Unlike previous models, our dataset exceeds 1.5T tokens for pre-training phase. Second, we redesign the model architecture: rather than finetuning an English-dominant model (which risks overwriting existing knowledge), we upscale a smaller open model to 75B parameters and define a new architecture, providing dedicated capacity for Thai specialization. Through these two innovations, we progressively decouple from upstream open-source backbones and gain full control over cultural and knowledge biases present in the final model.

Extensive evaluations demonstrate JAI-1’s superiority in Thai-language benchmarks, including Thai-MT-Bench and Thai-IFEval, alongside robust performance in cultural understanding tasks. By bridging the representation gap and optimizing for localized utility, this work advances the development of culturally aware, efficient LLMs for underrepresented linguistic communities.

2 JAI-1 Overview

2.1 Upscaling Strategy for Architecture Design

The JAI-1 project aims to develop a Thai-language-specific LLM capable of deep understanding of Thai language and cultural contexts through extensive training on Thai datasets. Central to this endeavor is the principle that foundational intellectual capabilities—such as general knowledge, reasoning, and multi-task proficiency—must be established before enhancing Thai-specific competencies. To achieve this, we adopt an up-scaling strategy: starting with a smaller, knowledge-rich base model, we expand its architecture to integrate additional linguistic and cultural knowledge, thereby creating a high-performing Thai-centric LLM. This approach bypasses the inefficiencies of training from scratch by leveraging pre-trained open-source models as a robust starting point [13, 14].

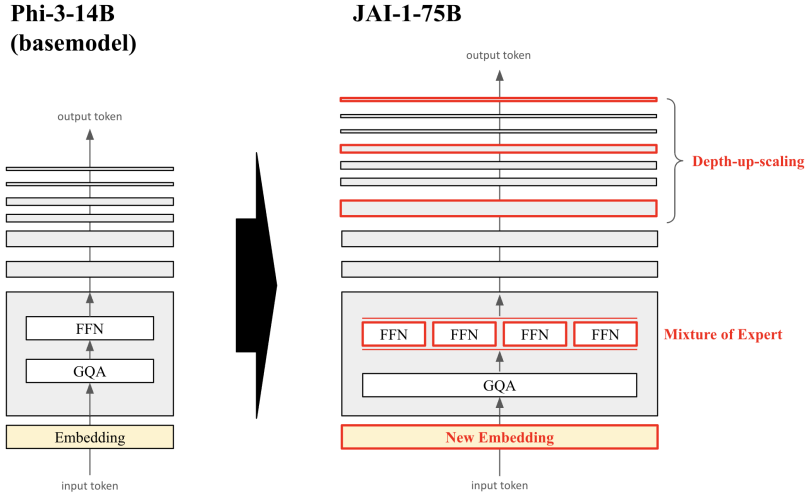


Figure 1: Architectural overview of the JAI-1 (75B) model. JAI-1 is a Thai-optimized LLM developed by scaling the Phi-3-medium (14B) architecture. While Phi-3-medium excels in general knowledge benchmarks, JAI-1 significantly enhances its parameter count to 75B through depth-up-scaling (DUS) and mixture-of-experts (MoE) techniques. Additionally, the model incorporates a redesigned Thai-optimized tokenizer to improve representation of Thai script and linguistic features. In the figure, GQA and FFN represent grouped-query attention and feed-forward network layers respectively.

To comply with licensing requirements, we evaluated open-source models with permissive terms. In April 2024, Microsoft’s phi-3-medium (14B) [15] emerged as a compelling candidate. Despite its compact architecture, it exhibited robust foundational capabilities, achieving a MMLU score of 78.0—notably competitive with larger models like Llama-3-70B-Instruct (80.2) and GPT-4

Turbo (84.0 While these larger models prioritize raw performance, they lacked the efficiency and adaptability required for our Thai-focused use case. The phi-3-medium’s optimal balance of size and performance made it a strategic choice for scaling our system, particularly for deployment in resource-constrained environments.

To scale phi-3-medium to 75B parameters—over five times increase—we implemented three core strategies:

- **Tokenizer Adaptation:** The model’s tokenizer was redesigned to optimize processing of Thai script, addressing linguistic nuances such as tonal markers and compound words
- **Depth-Up-Scaling (DUS) [14]:** The architecture was expanded through additional transformer layers and parameter increases, enhancing its capacity for complex reasoning and knowledge retention.
- **Mixture-of-Experts (MoE) [16]:** MoE layers were integrated to enable dynamic task specialization, allowing subsets of the model to focus on Thai-specific challenges like idiomatic expressions and cultural references.

This strategic expansion preserved the base model’s foundational knowledge while introducing Thai-centric enhancements. The upscaling process was visualized in Figure 1, which outlines architectural modifications.

2.2 Training Pipeline

Following the architectural modifications, the JAI-1 model underwent a standard large language model (LLM) training pipeline comprising pre-training and post-training phases. The pre-training phase focused on equipping the model with general linguistic and reasoning capabilities through next token prediction on large-scale raw Thai-language datasets. This stage enabled the model to acquire core knowledge about syntax, semantics, and world phenomena, forming the bedrock for subsequent task-specific refinements. The post-training phase was divided into two distinct sub-stages to align the model with practical use cases:

- **Supervised Finetuning (SFT):** The model was trained on high-quality datasets formatted with chat templates, teaching it to follow instructions and generate contextually appropriate responses.
- **Alignment Tuning:** This stage introduced learning from human feedback to refine the model’s behavior.

By structuring post-training into SFT and alignment phases, JAI-1 achieved both task proficiency and value-aligned behavior, critical for deployment in real-world applications. The following sections will provide more detailed explanations for pre-training, supervised finetuning and alignment tuning.

3 JAI-1 Architecture

As described in Section 2.1, we applied three upscaling strategies: tokenizer adaptation, DUS and MoE. This section describes each upscaling approach and the finally identified model configuration.

3.1 Upscaling Strategies

Publicly available pre-trained language models (PLMs) are highly optimized for specific data distributions, which often leads to challenges when performing additional continual pre-training (CPT). These challenges include: (1) difficulties in smoothly integrating new knowledge [17] and (2) catastrophic forgetting [12], where previously acquired knowledge is lost. Since we plan to inject a large volume of Thai language data, we employ an upscaling strategy to ensure the model parameters can accommodate additional Thai linguistic information and knowledge. Specifically, we implement three key approaches: Tokenizer adaptation, DUS and MoE. This subsection details each of these methods.

3.1.1 Tokenizer Adaptation

Phi-3-medium employs a character-level Byte Pair Encoding (BPE) tokenizer with a vocabulary size of 32,064 tokens, which is primarily optimized for English subwords. While the model supports multilingual processing, its original vocabulary lacks sufficient coverage for Thai-specific linguistic patterns. To address this limitation, we expanded the tokenizer by incorporating Thai-language corpora into the BPE training process. This extension increased the total vocabulary size to 64,000 tokens, enhancing the model’s ability to represent Thai character sequences effectively. Detailed principles of BPE tokenization are referenced in [18].

The expansion of the vocabulary introduced a critical challenge: the original 32,064 token embeddings in phi-3-medium could not accommodate the additional 31,936 tokens. Randomly initializing these new embeddings risks destabilizing training by introducing noisy parameters and increasing computational costs. To mitigate this, we propose a token embedding merging strategy that leverages semantic relationships between new and existing tokens. The method operates in two steps:

- **Origin Token Matching:** Each new token is decomposed into constituent subwords using the original tokenizer, yielding a set of existing (origin) tokens.
- **Embedding Aggregation:** The embeddings of these origin tokens are averaged to initialize the new token’s embedding, enabling semantically grounded initialization rather than random assignment.

This approach preserves semantic consistency for 64,000 tokens, while only a negligible fraction (e.g., tokens with no matching origin subwords) required

random initialization. By reducing reliance on arbitrary parameter values, this method enhances training stability and efficiency when adapting the model to new linguistic domains.

To quantify the impact of our adaptation on Thai-text token efficiency, we measured the total token count and average tokens per character (TPC) using the output of the training split from the WangchanThaiInstruct corpus¹. Table 1 reports the results for our adapted tokenizer (JAI), the original Phi-3, and other state-of-the-art multilingual baselines.

Tokenizer	Total Tokens	TPC
JAI	13,436,003	0.2913
phi-3	47,562,113	1.0311
phi-4	18,914,748	0.4100
Qwen-3	25,328,006	0.5491
Gemma-3	15,763,728	0.3417
Llama-3	21,454,143	0.4651

Table 1: Token efficiency on Thai text: lower TPC is better.

As shown, our adapted JAI tokenizer achieves a TPC of 0.2913, over a 71% reduction compared to the original Phi-3, and outperforms all multilingual baselines in compactness. This demonstrates that incorporating Thai-specific BPE rules yields a substantially more efficient token representation.

3.1.2 Advanced Depth-Up-Scaling

DUS is a technique that increases model depth by duplicating existing layers, reducing the computational cost of training from scratch. The original DUS method merges K layers from the input side and K layers from output side to build a deeper network with 2*K layers. In this work, we applied standard DUS to extend a 40-layer phi-3-medium model to 64 layers by setting K=32. However, unlike with Llama-based models [14], we observed significant instability in the early training phase when applying this method to the phi-3-medium architecture.

Since depth-scaling via DUS can be achieved through various strategies beyond the original method, we experimented with multiple configurations and identified two key structural improvements that led to DUS.v2: Block-level Duplication and Skip Connections Between Identical Blocks.

- **Block-level Duplication:** Unlike standard DUS, which duplicates individual layers, our approach organizes the entire model into blocks and replicates selected blocks in their original positions.
- **Skip Connections Between Identical Blocks:** In the original model, information flows through blocks in a specific sequence. Block-level Du-

¹<https://huggingface.co/datasets/airesearch/WangchanThaiInstruct>

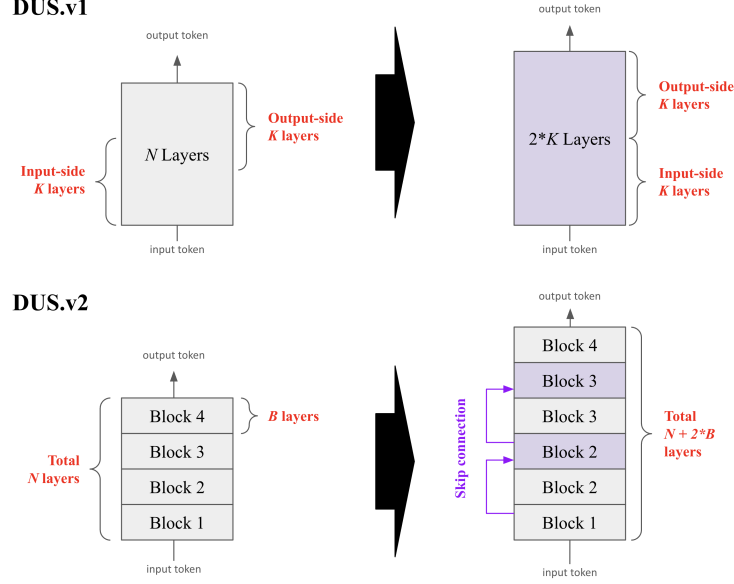


Figure 2: Visual illustration of the key differences between two depth-scaling strategies in the DUS framework. The original DUS method, referred to as DUS.v1, extends model depth by selecting K layers from the input side and K layers from the output side, then concatenating these two sets to form a deeper architecture with $2K$ layers. In contrast, DUS.v2 introduces a block-level approach: the original model is divided into multiple blocks, each containing B consecutive layers, and depth is scaled by duplicating selected blocks in their original sequential order.

plication disrupts this flow by repeating blocks, which can hinder training efficiency and performance. To mitigate this, we introduced additional skip connections between duplicated blocks. These connections preserve the original information-processing pathways, allowing the model to leverage learned representations more effectively.

Figure 2 provides a visual explanation of these two duplication strategies of DUS.v1 and DUS.v2. The most significant distinction between the two methods lies in how they preserve the layer order of the base model. In DUS.v1, the two defined blocks (input-side and output-side) retain the original layer sequence of the base model. The only disruption to the base model’s layer order occurs at the single merging point where the two blocks are concatenated, which explains why DUS.v1 often leads to suboptimal performance without additional fine-tuning. While this simplistic block definition and merging strategy is straightforward, it lacks flexibility to accommodate diverse DUS strategies across different model architectures.

In contrast, DUS.v2 first divides the base model’s layers into fixed-size

blocks, then selectively duplicates chosen blocks to scale depth. Though this approach offers greater flexibility for depth scaling, it introduces more disruptions to the original layer sequence compared to DUS.v1. For instance, in Figure 2, the repetition of Block 2 and Block 3 creates points where the original information flow is altered, potentially affecting model performance.

To address disruptions in the base model’s information flow caused by block duplication, DUS.v2 incorporates additional skip connections. As shown in Figure 2, when a block is duplicated (e.g., Block 2 or Block 3 in purple), a skip connection is established from the previous block in the original sequence. This can be expressed mathematically as:

$$\text{Output}_{n,(d)} = \text{Block}_n(x + \alpha_{n,(d)} * \text{Output}_{n-1,(\max d)}) \quad (1)$$

where n indicates the block index of the base model (origin) and d represents the duplication index of the DUS model. $\text{Output}_{n,(d)}$ is the output of the d -th duplicated block of Block_n . For examples, the output of the first Block 2 in grey is represented as $\text{Output}_{2,(1)}$ and the output of the second Block 2 in purple is denoted as $\text{Output}_{2,(2)}$. x is the input feature from the previous layer in the DUS model and $\alpha_{n,(d)}$ is a scaling factor for the original information process. $\max d$ indicates the latest duplication index. Importantly, $\alpha_{n,(1)} = 0$ for the first block in each duplication sequence, as no skip connection is needed before duplication begins.

This approach preserves the base model’s inherent information flow within the duplicated blocks. For example, in Figure 2, the duplicated Block 2 and Block 3 receive skip connections from the original Block 1 and Block 2, respectively. This mimics the base model’s original flow (Block 1 $\rightarrow$ Block 2 $\rightarrow$ Block 3), ensuring continuity even after depth scaling. Internal evaluations confirmed that this modification significantly improves training stability and learning efficiency post-DUS, contributing to more consistent performance gains.

blocks	origin layers	block usage	# of layers after DUS.v2
Block-1	layer1-4	1	4
Block-2	layer5-8	1	4
Block-3	layer9-12	1	4
Block-4	layer13-16	2	8
Block-5	layer17-20	3	12
Block-6	layer21-24	3	12
Block-7	layer25-28	2	8
Block-8	layer29-32	1	4
Block-9	layer31-36	1	4
Block-10	layer37-40	1	4

Table 2: Block-level duplication applied to JAI-1. Block1-3 and Block 8-10 are not duplicated in the DUS process. The middle blocks are duplicated for 1 or 2 times. Finally, the number of layers of the model after DUS.v2 becomes 64.

In JAI-1, we empirically determined that a block size of 4 layers was optimal. We excluded some input-side and output-side blocks from replication to scale the model’s depth while maintaining structural integrity. Table 2 illustrates which blocks are replicated and how many times.

3.1.3 Mixture of Expert Design

Motivated by advancements in models like Mistral-8x22B-v0.1 and Phi-3.5-MoE, we incorporate a basic MoE framework into JAI-1. The MoE architecture is structured as follows: (1) At each transformer layer, $M=4$ experts (feed-forward networks, FFNs) are defined and (2) A routing layer selects $K=2$ active experts before processing inputs through the FFN. The outputs from the selected experts are then aggregated. For a comprehensive overview of MoE principles, refer to [16].

A key challenge in MoE initialization arises when one expert is initialized with pretrained FFN weights (from DUS.v2), while the remaining three are randomly initialized. This asymmetry can introduce routing bias, where the model disproportionately favors the pretrained expert. To mitigate this, we employ a heuristic strategy inspired by weight-space exploration:

- **Pretraining and Snapshot Collection:** We first train the non-MoE version of JAI-1 for 6K steps. During this phase, we save 3 snapshots of the FFN weights at intervals of 2K steps (e.g., steps 2K, 4K, 6K).
- **Expert Initialization:** The initial (step 0) FFN weights are assigned to Expert 0. The 3 snapshots are used to initialize Experts 1-3, avoiding random initialization.

By leveraging learned weights from different training stages, we reduce routing bias and ensure balanced expert selection. This approach avoids introducing random noise into the weight space while identifying viable expert candidates.

3.2 Architecture Configuration

Here, we introduce the final JAI-1 architectural configuration after applying three upscaling methods. Since expanding phi-3-medium, JAI-1 retains its origin layer design: embedding size, intermediate size, activation method, normalization method and the number of attention heads. However, modifications from the tokenizer adaptation, DUS and MoE, increase vocabulary size, the number of layers and the number of experts, collectively achieving a parameter scale-up of over 5x. Table 3 presents the model architecture configurations of phi-3-medium and JAI-1. As shown, while most design options are shared, differences exist in terms of model size.

During the model scaling process, we additionally decided to eliminate the sliding window constraint inherent to phi-3-medium. While a fixed 2K sliding window can improve memory efficiency during serving, it poses challenges in achieving robust performance for long contexts. We identified this limitation in

	Attribute	phi-3-medium	JAI-1
Model size	Parameter size	14.0B	75.3B
	Embedding size	5120	5120
	Intermediate size	17920	17920
	# of experts	1	4
	# of layers	40	64
	Vocabulary size	32064	64000
Layer design	Positional encoding	RoPE($\theta=10K$) [19]	RoPE($\theta=1M$) [19]
	Sliding window	2047	Null
	Activation	SiLU [20]	SiLU [20]
	Normalization	RMSNorm [21]	RMSNorm [21]
	Attention layer	GQA [22]	GQA [22]
	# of heads	40	40

Table 3: Architecture configuration of JAI-1. JAI-1 adopts an up-scaling strategy to expand the phi-3-medium base model, increasing vocabulary size, layers, and experts to achieve a total parameter count of 75.3B. While retaining core architectural designs from phi-3-medium, key modifications include adjustments to the sliding window mechanism and RoPE parameters to enhance long-context handling and future token length scalability.

phi-3-medium and, during the scale-up, removed this constraint. Instead, we increased the RoPE [19] parameter θ to adopt a positional encoding method better suited for long-context scenarios.

4 Pre-training

The primary challenges in pre-training involve (1) collecting and curating a large-scale Thai language dataset and (2) integrating Thai data without compromising the model’s performance. This section provides an overview of the corpora utilized for JAI-1 training, details the pre-training strategies applied, and presents the outcomes of the pre-training process.

4.1 Thai Corpora for pre-training

In this project, we constructed a Thai corpus comprising over 300 billion (B) tokens for pre-training. The corpus is categorized into four distinct groups based on its composition method:

- **Public and open data:** We collected publicly available, pre-processed Thai language data, totaling 61B tokens. During this process, datasets deemed to have excessively low quality were excluded. Statistics and sources of the collected data are detailed in Table 4.

- **Private data:** This dataset was provided by our partners. It consists of 4B billion tokens that are collected from internet sources that are not publicly available or free.
- **Web-crawled data:** Collected manually using a crawling engine, this dataset required site-specific parsers developed individually for each site. Although high-priority data was prioritized, the total volume remained relatively small at 0.5B tokens. Due to the scale limitations, additional collection was paused after initial development. Statistics and sources are detailed in Table 5.
- **Upstage’s processed data:** This category includes two subsets: translated high-quality data and LP-pipeline-based Common Crawl data. The former leverages Upstage’s proprietary data and translation methods. The latter utilizes Upstage’s LP data pipeline [23] to process a large-scaled Common Crawl dump data. Both underwent deduplication and rule-based filtering. The total size is 239.3B tokens, with statistics and sources listed in Table 6

In sum, the pre-training dataset consists of $165 + 174 = 340$ B organic tokens and 65B synthetic tokens, totaling 405B tokens.

data name	tokens	source
thaigov-corpus	0.06B	Thai government website since 2018.02 ²
th-law	0.10B	Office of the council of state, Thailand ³
th-tcc	0.00B	Office of the council of state since 1932 ⁴
th-mC4	12.53B	cleaned version of Common Crawl’s corpus ⁵
th-oscar	11.18B	web-based multilingual resources and datasets ⁶
th-xlsum	0.01B	article-summary pairs from BBC ⁷
th-sum	0.21B	Thai online news ⁸
th-culturaX	17.36B	culturaX dataset ⁹
th-wiki	0.13B	Thai Wikipedia ¹⁰
th-TLC	2.36B	Vajirayana Digital Library ¹¹
th-HPLT-v1.2	17.17B	monolingual and bilingual corpus ¹²

Table 4: Public and open Thai corpus used in JAI-1. The total number of tokens from this data is 61B.

²<https://github.com/PyThaiNLP/thaigov-corpus>

³<https://github.com/PyThaiNLP/thai-law>

⁴<https://github.com/PyThaiNLP/Thai-constitution-corpus>

⁵<https://huggingface.co/datasets/legacy-datasets/mc4>

⁶<https://huggingface.co/datasets/oscar-corpus/OSCAR-2301>

⁷<https://huggingface.co/datasets/csebuethlp/xlsum>

⁸<https://huggingface.co/datasets/thaisum>

⁹<https://huggingface.co/datasets/uonlp/CulturaX>

¹⁰<https://huggingface.co/datasets/pythainlp/thai-wiki-dataset-v3/viewer>

¹¹<https://attapol.github.io/tlc.html>

¹²<https://hplt-project.org/datasets/v1.2>

data name	tokens	source
th-bangkokbiz	0.104B	https://bangkokbiznews.com
th-rath	0.358B	https://thairath.co.th
th-blognone	0.002B	https://blognone.com
th-deckd	0.002B	https://deck-d.com
th-mhesi	0.003B	http://mhesi.go.th

Table 5: Crawled data from this project. The total number tokens is about 0.5B. We listed up more crawling sites but found that this was not scalable. We decided to turn to translation of our own corpus and filtering process from a large scaled open-sourced data form Common Crawl.

data name	tokens	source
th-upstage-translation	65.30B	Translate Upstage’s corpus into Thai.
th-lp-pipeline	174.00B	LP pipeline from Common Crawl [23]

Table 6: Private Thai corpus developed in the project. The total number tokens is about 239.3B.

4.2 Pre-training Strategy

The JAI-1 architecture is an upscaled version of a 14B model, requiring dual objectives during training: recovering performance lost during upscaling while enhancing baseline capabilities, and integrating Thai language data. To achieve this, we divided the training into three distinct phases: (1) English-centric pre-training, (2) Thai data injection, and (3) context window expansion.

- **Phase 1. English-Centric Pre-training:** This phase focused on restoring performance degraded during upscaling and further strengthening baseline capabilities. A key challenge was curating English datasets to maintain or recover the base model’s performance level of 78 points (MMLU score). The base model was highly optimized on proprietary, high-quality datasets that were not publicly available. After upscaling, the initial MMLU score dropped to 62 points due to architectural changes, necessitating the development of a tailored data recipe to restore it to the original 78 points.
- **Phase 2. Thai Data Injection:** In this phase, we integrated the optimized data recipe from Phase 1 with Thai language datasets (defined in Section 4.1) to prevent catastrophic forgetting. The training data distribution was balanced to allocate 27% of the total dataset to Thai corpus, with the remaining 73% consisting of multilingual data (primarily English) to preserve the English performance gains from Phase 1.
- **Phase 3. Context Window Expansion:** The base model natively supported a 4K context window, which was preserved during Phases 1 and

2 for performance stability. In the final phase, we expanded the context window to 32K using the sequence-packing method [24] on the existing data recipe.

The model underwent a total of 1.5T tokens of pre-training across all phases. The final PLM met the project’s objectives, achieving strong performance in both English and Thai benchmarks, including Thai Exam evaluations.

5 Pre-trained LLM Evaluation

5.1 Evaluation Benchmarks for Pre-trained Models

To assess the model’s baseline intelligence, we adopted a logit-based evaluation method provided by im-eval-harness, focusing on multiple-choice formats. Specifically, this approach involves presenting the model with predefined answer options and measuring accuracy by evaluating the likelihood of selecting the correct choice. For further details, please refer to [25] and the im-eval-harness repository¹³.

We defined three key metrics to track model performance at this evaluation stage: Eng-H6, Thai-H6, and Thai-Exam.

- **Eng-H6:** A consolidated benchmark suite widely used for LLM evaluation, integrating six datasets: MMLU [26], ARC [27], HellaSwag [28], TruthfulQA [29], WinoGrande [30], and GSM8K [31]. Performance is measured by averaging individual scores across these datasets. This metric serves as a standard for evaluating foundational English-language knowledge.
- **Thai-H6:** A Thai-translated version of the Eng-H6 datasets. The six benchmarks were translated into Thai and refined through human verification to ensure accuracy.
- **Thai-Exam:** A Thai knowledge benchmarking dataset comprising multiple-choice questions sourced from Thai examinations¹⁴. This dataset was rapidly integrated into our project for evaluation purposes.

5.2 Evaluation Results on Pre-trained Models

Table 7 compares model performance across the Eng-H6, Thai-H6, and Thai-Exam benchmarks. The first key observation is the performance improvement achieved by upscaling and fine-tuning the phi-3-medium [15] model. While phi-3-medium initially showed weak results on Eng-H6 and Thai-Exam, the upgraded version demonstrated significant gains in both metrics. The second critical comparison is against GPT-3.5, which was our initial performance target across all three benchmarks. Our model achieved performance on par with

¹³<https://github.com/EleutherAI/lm-evaluation-harness>

¹⁴https://huggingface.co/datasets/scb10x/thai_exam

	Models	Eng-H6	Thai-H6	Thai-Exam
Our baseline	Phi-3-medium	62.89	-	35.02
	GPT-3.5 (<i>objective</i>)	71.00	71.65	46.00
Thai LLMs	Typhoon-v1.5-72B	-	64.80	61.70
	OpenThaiGPT-1.5-72B	-	79.69	-
Ours	JTS-1-PLM	78.95	71.80	62.32

Table 7: Performance comparison on Eng-H6, Thai-H6, and Thai-Exam. GPT-3.5 was the initial objective of this project and JAI-1-PLM achieves better performance than the objective.

or exceeding that of GPT-3.5 in these metrics. Additionally, we compared our model to Typhoon-v1.5-72B [6], a model released in May 2024 that was trained by adding Thai data to Qwen1.5-72B [10]. When evaluating Thai-specific metrics (Thai-H6 and Thai-Exam), JTS-1-PLM outperformed Typhoon-v1.5-72B. Finally, we conducted a comparison with OpenThaiGPT-1.5-72B, released in September 2024. This model appears to be built on Qwen2.5-72B [32] with additional Thai data training. Due to the stronger foundation of the Qwen2.5-based architecture, OpenThaiGPT-1.5-72B achieved better Thai-H6 performance than JTS-1-PLM.

6 Post-training: Supervised Finetuning

The Supervised Finetuning (SFT) stage is crucial for enabling Pre-trained Language Models (PLMs), initially capable only of Next Token Prediction, to perform effectively in general-use conversational scenarios. High-quality training data is essential to achieve successful outcomes in this stage. Although open-source datasets for supervised finetuning have been made available, directly using these datasets presents two significant challenges: (1) many are low-quality synthetic data produced by earlier-generation models such as ChatGPT-3.5, and (2) they typically lack sufficient Thai language content.

To address these limitations, we employed a data augmentation strategy, using existing open-source data as seeds for regeneration, thereby enhancing data quality. Additionally, we developed specialized models—an English-to-Thai Translator and a Thai Fluency Reward model—to effectively extend English datasets into Thai. Moreover, we synthesized data specifically designed to improve understanding of Thai cultural nuances, further enriching our training set and resulting in a robust Thai-focused language model.

This session is structured as follows: (1) Data Generation Methods for each category, (2) Model Training Procedures, and (3) Analysis of Training Outcomes.

6.1 Eng-Thai Translator and Thai Fluency Reward Model

6.1.1 Eng-Thai Translator

To leverage extensive and diverse English datasets for SFT, we developed an English-to-Thai translation model. The foundational model was Upstage’s multilingual internal 10.7B LLM, which we finetuned for translation purposes. The training dataset comprised 0.5M English-Thai pairs from various sources, including large-scale open-source resources, news/media, government/legal texts, and web crawling.

This model outperformed GPT-3.5-turbo and showed comparable performance to GPT-4o-2024-05-13, achieving superior results on MMLU and ARC tasks but slightly lower performance on Flores. Despite its modest size and resource constraints, this translator significantly contributed to generating high-quality Thai data.

Model	FLORES	ARC	MMLU
GPT-4o-2024-05-13	46.5	62.7	72.2
GPT-4-0125-preview	41.3	63.0	64.6
Solar-10.7B-translation-enth	41.6	66.6	73.5

Table 8: English to Thai translation performance comparison. The measurements are BLUE scores on FLORES, ARC and MMLU.

6.1.2 Thai Fluency Reward Model

To enhance the quality of synthetic Thai data, we developed a Thai Fluency Reward model. The foundational model was Solar Mini 1.2, a multilingual internal model from Upstage, which we finetuned using specially constructed fluency assessment datasets. The resulting reward model was then utilized for data validation.

Specifically, the reward model consisted of a 10.7B-sized LLM augmented with a linear layer capable of reward scoring. Fluency scores were generated using GPT-4o from diverse chat datasets, and pair-wise ranking data were constructed. Training was performed using the pair-wise ulytrainteract loss method, achieving approximately 90% fluency classification accuracy.

We applied this model to evaluate and filter generated Thai data based on logit values, employing a systematic thresholding approach. The final threshold value selected for filtering was 0.88.

6.2 Supervised Finetuning Data

Our comprehensive SFT dataset comprises six primary categories: (1) General Chat Ability, (2) Instruction Following, (3) Wild Task Ability, (4) Thai Cultural Understanding, (5) Thai Safety, and (6) Long Context Understanding.

Each category employed distinct data generation strategies, with Thai-language datasets undergoing additional reward-based filtering to ensure quality before their inclusion in training.

Category	Language	Seed	# of examples	# of tokens
1.General Chat	EN, TH	Capybara	65,664	70.0M
2.Instruct Following	EN, TH	Upstage’s	185,270	154.6M
3.Task Ability	EN, TH	flan_v2, Upstage’s	139,986	204.5M
4.Thai Culture	TH	Wikipedia	169,276	22.5M
5.Thai Safety	TH	-	5,235	0.5M
6.Long Context	EN	Upstage’s	13,369	158.4M
Total			578,800	610.2M

Table 9: SFT Data Statistics.

6.2.1 General Chat Ability

The simplest application of an LLM is general user chitchat. We utilized the widely adopted open-source Capybara dataset to construct this dataset. However, since Capybara only contained English data, we translated the data into Thai and refined it using OpenAI models to better align with Thai cultural contexts. Given the low complexity and broad accessibility of general knowledge, the data creation process remained relatively straightforward.

6.2.2 Instruction Following

Open-source datasets specifically for instruction following are limited. To overcome this, we leveraged the English instruction following dataset owned by Upstage, which was generated by synthesizing multi-turn instruction examples from an instruction pool using Upstage’s proprietary models. However, this dataset was exclusively English, lacking Thai-specific instruction following data, and suitable open-source LLMs for synthesis were unavailable.

To resolve this, we translated the instruction pool into Thai and synthesized new data using GPT-4o, incorporating a filtering logic to ensure appropriate Thai language generation.

6.2.3 Wild Task Ability

LLMs have broad potential applications. To expand the range of tasks LLMs can perform, we used datasets derived from Flan. However, these datasets lacked Thai data, limiting model performance. Directly utilizing Flan datasets proved suboptimal due to quality issues. Therefore, we refined the data by regenerating queries and answers using advanced open-source models, translating queries into Thai, and regenerating Thai answers accordingly.

Recognizing from MT-Bench evaluations that Coding, Math, Reasoning, and Extraction tasks required further improvement, we created additional specialized datasets based on queries from Upstage’s repositories. These datasets were translated and answers were generated using open-source models.

6.2.4 Thai Cultural Understanding

A Thai-focused LLM must extend beyond mere language proficiency to include an understanding of Thai cultural and historical contexts. We synthesized data using content from Thai Wikipedia, ensuring the model gained insights into Thai cultural nuances. Furthermore, we added specialized data about the Jasmine group to enhance cultural understanding.

6.2.5 Thai Safety

To manage sensitive information within Thai cultural contexts, we created queries involving sensitive topics and generated synthetic refusal responses, ensuring the model appropriately handles sensitive scenarios.

6.2.6 Long Context Understanding

Finally, to extend context handling capabilities, we developed long-context document-based QA data, thereby enhancing the model’s performance on RULER benchmarks.

6.3 Supervised Finetuning Stages

We conducted experiments using the assembled datasets to identify the optimal combination of training strategies. Preliminary experiments showed that training with individual datasets provided measurable improvements, but due to differences in task complexity and data diversity, it was necessary to determine an optimal training ratio in an integrated setting.

To address this, we designed multiple “recipes” that controlled the proportions of different dataset categories (Chat, Instruction Following (IF), Task, Culture, Safety, and Long-form data). Table 10 summarizes two of the most effective recipes. For example, SFT.1 allocated 39.32% of its training data to cultural datasets and 24.72% to task-based data, whereas SFT.2 placed greater emphasis on IF data (32.01%) while reducing the proportion of cultural data (29.25%). The inclusion of safety-related examples was kept deliberately small in both cases (2%) to prevent overfitting while still instilling baseline alignment behavior. These variations allowed us to probe how shifts in balance between instruction-following and cultural data would affect downstream performance.

Benchmark evaluations of the resulting models are reported in Table 11. Both recipes achieved competitive results across Thai benchmarks, but with differing strengths. SFT.1 achieved the best scores on IFEval-TH (77.00) and JAI-Hall-Bench (63.20), demonstrating strong reasoning and reduced hallucination. In contrast, SFT.2 achieved the highest score on MT-Bench-TH (7.45),

suggesting better multi-turn conversational performance. To exploit the complementary strengths of the two recipes, we created a merged model by averaging their weights. The merged model produced a balanced outcome across all metrics: IFEval-TH (74.96), MT-Bench-TH (7.33), and JAI-Hall-Bench (62.10). Although not the single best on any individual benchmark, this merged model exhibited more stable and robust performance overall.

Through these experiments, we identified that both recipe design and post-hoc model merging are viable strategies for maximizing performance. The evidence suggests that carefully tuning dataset composition influences different capability axes (instruction-following, cultural grounding, and hallucination control). Moreover, merging independently trained models provided a practical method for reconciling trade-offs between these axes. After testing various ratios, we determined that a simple 1:1 weight merging of SFT.1 and SFT.2 was optimal, offering a well-rounded model suitable for deployment in diverse Thai NLP applications.

Recipe	Chat	IF	Task	Culture	Safety	Long
SFT.1	10.15%	24.19%	24.72%	39.32%	1.62%	0.00%
SFT.2	11.34%	32.01%	24.19%	29.25%	0.90%	0.00%

Table 10: SFT data balance recipes. We evaluated more than 10 recipes and found that the two showed the best performance.

Model	IFEval-TH	MT-Bench-TH	JAI-Hall-Bench*
SFT.1	77.00	7.18	63.20
SFT.2	73.00	7.45	56.00
Merged	<u>74.96</u>	<u>7.33</u>	<u>62.10</u>

Table 11: Thai benchmark performance of SFT models. SFT.1 and SFT.2 are independently trained from the same pre-trained model, while Merged refers to a model obtained by averaging the weights of SFT.1 and SFT.2. Benchmark details are provided in Section 6.1. JAI-Hall-Bench* denotes an earlier version of the benchmark used for the final evaluation; it includes 298 samples excluding Wiki QA and focuses solely on hallucination detection without assessing fluency.

7 Post-training: Alignment-tuning

This stage aims to incorporate human preferences into the model’s behavior. We employed Direct Preference Optimization (DPO) to perform alignment tuning and curated datasets across four critical dimensions: (1) chat alignment, (2) instruction-following alignment, (3) fluency alignment, and (4) hallucina-

tion alignment. These datasets were used to train the model to generate more preferred and reliable responses.

7.1 Alignment-tuning Method

Various methods have been proposed to align large language models (LLMs) with human preferences, beginning with Reinforcement Learning from Human Feedback (RLHF). In this work, we adopt Direct Preference Optimization (DPO), an offline reinforcement learning method, as an effective and computationally efficient approach for alignment tuning. DPO enables the model to learn directly from pairwise preference data without requiring an explicit reward model or costly online rollouts.

7.2 Alignment-tuning Data

The alignment tuning process was supported by four types of datasets, each tailored to improve a specific aspect of model alignment: chat alignment, Instruction Following Alignment, Fluency Alignment and Hallucination Alignment (Table 12).

Category	Language	Seed	# of examples
1.General Chat	TH	Capybara	7,893
2.Instruct Following	EN, TH	Upstage’s	67,462
3.Thai Hallucination	TH	Wikipedia	4,456
Total			79,811

Table 12: DPO Data Statistics. All examples have chosen and rejected response pairs.

7.2.1 General chat alignment

To ensure that the model provides responses preferred by human users during open-ended conversations, we fine-tuned the model using pairwise preference data. Starting from a model trained via supervised fine-tuning (SFT), we generated five candidate responses per prompt. These responses were evaluated using GPT-4o (2024-11-20), and samples receiving high preference scores were labeled as positive while those with low scores were labeled as negative. This data was then used to train the model using DPO. The seed data for this process was derived from Thai-language data generated based on the Capybara dataset. A total of 7,893 pairwise samples were constructed.

7.2.2 Instruction following alignment

To enhance the model’s ability to follow user instructions in Thai, we analyzed failure cases encountered during data generation. From these, we curated

positive and negative response pairs based on performance deviations. This instruction-following alignment dataset consists of 67,462 pairwise samples.

7.2.3 Hallucination alignment

To address factual correctness, we used a Thai Wikipedia QA dataset and generated answers using the trained model. These outputs were evaluated by Claude to detect hallucinations. Responses judged to contain hallucinations were labeled as negative, while those without were labeled as positive. This process yielded a dataset specifically designed to reduce hallucinations during generation.

8 Evaluation for Instruct Models

8.1 Evaluation Benchmarks for Instruct Models

To define the optimization objectives for our model development, we established four key benchmarks to comprehensively evaluate the final performance of the Chat model. Each benchmark focuses on a distinct capability crucial to high-quality language modeling, including instruction-following, multi-turn dialogue, cultural fluency, and long-context understanding.

- **IFEval-EN**: This benchmark assesses the instruction-following capabilities of Large Language Models. It is a benchmark focused on a set of verifiable instructions, such as specifying word counts or formatting requirements. The dataset is in English and is designed to evaluate chat or instruction-following fine-tuned language models.
- **IFEval-TH**: A Thai-translated and human-verified version of IFEval-EN. It measures how accurately LLMs follow verifiable instructions, with accuracy defined by whether the given instruction was correctly executed. This Thai version was created and released by the Typhoon project, which translated and validated the original IF-Eval samples.
- **MT-Bench-EN**: MT-Bench evaluates LLM performance in multi-turn dialogue scenarios across diverse domains. It consists of nine subcategories: Coding, Math, Reasoning, Extraction, Roleplay, Writing, Social Science, STEM, and Knowledge III. Model responses are assessed using the LLM-as-a-judge paradigm, with scores assigned on a 10-point scale.
- **MT-Bench-TH**: The Thai version of MT-Bench, which was translated and culturally adapted for the Thai language from the original MT-Bench-EN. We adopted the Thai version of MT-Bench-TH provided on the ThaiLLM leaderboard and used GPT-4o-2024-08-06 as the evaluation model, which was validated to provide stable results for benchmarking Typhoon2’s performance.

- **JAI-Hall-Bench:** Developed during the course of this project, this benchmark evaluates both cultural knowledge and Thai language fluency. It combines 298 project-specific samples and 1,000 QA items derived from Thai Wikipedia. Evaluation is based on two criteria: (1) the naturalness of the Thai-language response, and (2) the factual correctness of answers to Thailand-related questions. Both criteria are judged using the Claude-3.5-sonnet-20241022 model. A response is considered correct only if both aspects are rated as true. Final performance is reported as overall accuracy.
- **RULER:** RULER is a benchmark designed to evaluate long-context understanding in LLMs. It includes tasks such as retrieval, multi-hop tracing, and aggregation that go beyond simple information recall, requiring reasoning over extended input sequences. As context length increases, many models exhibit substantial performance degradation, highlighting the challenge of maintaining coherent reasoning in long-context settings.

8.2 Evaluation Results on Instruct Models

Models	IFEval EN	IFEval TH	MT-Bench EN	MT-Bench TH	JAI-Hall Bench	RULER
Typhoon2-70B [7]	86.7	81.0	82.2	73.6	50.0	95.4
JAI-1- α (v1.2)	-	69.0	-	64.0	21.0	92.0
JAI-1 (v1.3)	86.7	81.5	79.0	75.6	55.0	96.7

Table 13: Performance comparison between Typhoon2-70B and JAI-1. The initial version of JAI-1, JAI-1- α , shows poor performance when compared from Typhoon2-70B. However, the latest version of JAI-1 provides superior performance on all target benchmarks.

Table 13 presents a performance comparison across the benchmarks defined in Section 8.1. The primary comparison target and final model benchmark is Typhoon2-70B [7], a model released in December 2024 that builds upon the Llama3.3-70B architecture. Inheriting Llama’s foundational capabilities, Typhoon2-70B demonstrates strong performance on benchmarks like IFEval and MT-Bench. In our project, the earlier v1.2 model (JAI-1- α) consistently underperformed compared to Typhoon2-70B across all defined benchmarks. However, the final v1.3 model (JAI-1) surpassed Typhoon2-70B in all evaluation metrics, achieving superior performance across the board.

9 Conclusion

JAI-1 is a Thai LLM with 75B parameters, developed by upscaling the phi-3-medium (14B) model, which has strong English baseline performance, through tokenizer adaptation, DUS, and MoE techniques, resulting in a 5x increase in

model size. This approach expanded the parameter space by approximately 4x, enabling the storage of knowledge and information beyond English capabilities. By exploring optimized data recipes, we simultaneously improved performance in both English and Thai. Pre-training involved 1.5 trillion tokens, while post-training utilized over 600K examples. The pre-trained model achieved GPT-3.5-level performance on Eng-H6, Thai-H6, and Thai-Exam. The final JAI-1 model demonstrated superior performance compared to Typhoon2-70B on Thai-specific benchmarks like IFEval-TH, MT-Bench-TH, and JAI-Hall-Bench.

References

- [1] Muhammad Adilazuarda, Sagnik Mukherjee, Pradhyumna Lavania, Sidhant Singh, Alham Aji, Jacki O’Neill, Ashutosh Modi, and Monojit Choudhury. Towards measuring and modeling “culture” in llms: A survey. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 15763–15784, 2024.
- [2] Guangji Bai, Zheng Chai, Chen Ling, Shiyu Wang, Jiaying Lu, Nan Zhang, Tingwei Shi, Ziyang Yu, Mengdan Zhu, Yifei Zhang, et al. Beyond efficiency: A systematic survey of resource-efficient large language models. *arXiv preprint arXiv:2401.00625*, 2024.
- [3] Wen Yang, Junhong Wu, Chen Wang, Chengqing Zong, and Jiajun Zhang. Language imbalance driven rewarding for multilingual self-improving. *arXiv preprint arXiv:2410.08964*, 2024.
- [4] Peerat Limkonchotiwat, Kanruethai Masuk, Surapon Nonesung, Chalernpun Mai-On, Sarana Nutanong, Wuttikorn Ponwitararat, and Potsawee Manakul. Assessing thai dialect performance in llms with automatic benchmarks and human evaluation. *arXiv preprint arXiv:2504.05898*, 2025.
- [5] Xuan-Phi Nguyen, Wenxuan Zhang, Xin Li, Mahani Aljunied, Zhiqiang Hu, Chenhui Shen, Yew Ken Chia, Xingxuan Li, Jianyu Wang, Qingyu Tan, et al. Seallms-large language models for southeast asia. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, pages 294–304, 2024.
- [6] Kunat Pipatanakul, Phatrased Jirabovonvisut, Potsawee Manakul, Sittipong Sripaisarnmongkol, Ruangsak Patomwong, Pathomporn Chokchainant, and Kasima Tharnpipitchai. Typhoon: Thai large language models. *arXiv preprint arXiv:2312.13951*, 2023.
- [7] Kunat Pipatanakul, Potsawee Manakul, Natapong Nitarach, Warit Sirichotedumrong, Surapon Nonesung, Teetouch Jaknamon, Parinthapat Pengpun, Pittawat Taveekitworachai, Adisai Na-Thalang, Sittipong Sripaisarnmongkol, et al. Typhoon 2: A family of open text and multimodal thai large language models. *arXiv preprint arXiv:2412.13702*, 2024.

- [8] Sumeth Yuenyong, Kobkrit Viriyayudhakorn, Apivadee Piyatumrong, and Jillaphat Jaroenkantasima. Openthaigpt 1.5: A thai-centric open source large language model. *arXiv preprint arXiv:2411.07238*, 2024.
- [9] Medium. Typhoon 1.5 release. <https://medium.com/scb10x/typhoon-1-5-release-1dedf83c7959>.
- [10] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, et al. Qwen technical report. *arXiv preprint arXiv:2309.16609*, 2023.
- [11] Ronald Kemker, Marc McClure, Angelina Abitino, Tyler Hayes, and Christopher Kanan. Measuring catastrophic forgetting in neural networks. In *Proceedings of the AAAI conference on artificial intelligence*, volume 32, 2018.
- [12] Shezheng Song, Hao Xu, Jun Ma, Shasha Li, Long Peng, Qian Wan, Xiaodong Liu, and Jie Yu. How to alleviate catastrophic forgetting in llms finetuning? hierarchical layer-wise and element-wise regularization, 2025.
- [13] Jonathan Frankle and Michael Carbin. The lottery ticket hypothesis: Finding sparse, trainable neural networks. In *International Conference on Learning Representations*, 2018.
- [14] Sanghoon Kim, Dahyun Kim, Chanjun Park, Wonsung Lee, Wonho Song, Yunsu Kim, Hyeonwoo Kim, Yungi Kim, Hyeonju Lee, Jihoo Kim, et al. Solar 10.7 b: Scaling large language models with simple yet effective depth up-scaling. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 6: Industry Track)*, pages 23–35, 2024.
- [15] Marah Abdin, Jyoti Aneja, Hany Awadalla, Ahmed Awadallah, Ammar Ahmad Awan, Nguyen Bach, Amit Bahree, Arash Bakhtiari, Jianmin Bao, Harkirat Behl, et al. Phi-3 technical report: A highly capable language model locally on your phone. *arXiv preprint arXiv:2404.14219*, 2024.
- [16] Weilin Cai, Juyong Jiang, Fan Wang, Jing Tang, Sunghun Kim, and Jiayi Huang. A survey on mixture of experts in large language models. *IEEE Transactions on Knowledge and Data Engineering*, 2025.
- [17] Oded Ovadia, Menachem Brief, Moshik Mishaelli, and Oren Elisha. Fine-tuning or retrieval? comparing knowledge injection in llms. *arXiv preprint arXiv:2312.05934*, 2023.
- [18] Hugging Face. Byte-pair encoding tokenization. <https://huggingface.co/learn/llm-course/en/chapter6/5>, 2025. Detailed explanation of byte-level BPE, vocabulary building and merge rules.

- [19] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.
- [20] Stefan Elfving, Eiji Uchibe, and Kenji Doya. Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. *Neural networks*, 107:3–11, 2018.
- [21] Biao Zhang and Rico Sennrich. Root mean square layer normalization. *Advances in Neural Information Processing Systems*, 32, 2019.
- [22] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebron, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 4895–4901, 2023.
- [23] Yungi Kim, Hyunsoo Ha, Seonghoon Yang, Sukyung Lee, Jihoo Kim, and Chanjun Park. Lp data pipeline: Lightweight, purpose-driven data pipeline for large language models. *arXiv preprint arXiv:2411.11289*, 2024.
- [24] Hugging Face. Efficient llm pretraining: Packed sequences and masked attention. <https://huggingface.co/blog/sirluk/llm-sequence-packing>.
- [25] Medium. Llm evaluation — eleutherai lm-evaluation-harness. <https://medium.com/disasassembly/llm-evaluation-eleutherai-lm-evaluation-harness-cc379495d545>.
- [26] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.
- [27] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Øyvind Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge. In *arXiv preprint arXiv:1803.05457*, 2018.
- [28] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. Hellaswag: Can a machine really finish your sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 4791–4800, 2019.
- [29] Stephanie Lin, Jacob Hilton, and Owain Evans. Truthfulqa: Measuring how models mimic human falsehoods. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3214–3252, 2022.

- [30] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. Winogrande: An adversarial winograd schema challenge at scale. In *arXiv preprint arXiv:1907.10641*, 2019.
- [31] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Rei-ichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- [32] Qwen Team. Qwen2 technical report. *arXiv preprint arXiv:2412.15115*, 2024.